

Configuration-Driven Snapshot Training for Two-Head Intensity Prediction from Limit Order Books

BinanceDataTraining: System Specification (Internal)

February 16, 2026

Abstract

This document specifies a configuration-driven machine learning pipeline for predicting short-horizon price move intensities from limit order book data. The system builds disk-backed “snapshot” datasets by streaming order book rows from a time-series database, constructs fixed-shape multi-asset tensors using alignment policies and hybrid depth aggregation, and trains a convolutional–recurrent neural network with two classification heads that separately model upward and downward intensity. The emphasis is reproducibility: all behavioral choices are explicit in YAML configuration, validated by schema and semantic checks, and executed by a deterministic pipeline that logs artifacts and metrics to MLflow. We formalize the core transformations—time discretization, multi-asset alignment, hybrid aggregation, label construction, normalization, sample weighting, optimization, calibration, and backtesting—as equations aligned with the reference implementation.

1 Introduction

Electronic limit order books (LOBs) reveal a market’s instantaneous supply and demand through ranked bid and ask queues at multiple price levels. For short horizons, practical decision-making often requires more than predicting a binary direction; it requires estimating the intensity of plausible upward and downward excursions, which naturally leads to distributional targets over discrete movement bins. The system documented here operationalizes this view through a dual-head classification objective: at each prediction time, the model outputs a categorical distribution over upward intensity classes and, independently, a categorical distribution over downward intensity classes.

Two requirements shape the design. First, the representation must capture microstructural information at multiple depths while remaining stable under variable depth availability and database ingestion constraints. Second, the entire pipeline must be reproducible and auditable, so that changes in any element of data selection, feature construction, model architecture, or evaluation are reflected in versioned configuration rather than implicit code defaults. Consequently, all parameters are sourced from YAML, validated against a strict schema, and enforced by semantic constraints at startup.

The remainder of this document is organized into (i) methods and global architecture, focusing on the flow from configuration to evaluation, and (ii) detailed functionalities, mapping core implementation modules to mathematical formulations.

2 Methods and Global Architecture

2.1 Configuration-first execution model

The entry point (`main.py`) loads a YAML configuration and validates it against a schema (`config/validation_schema.yaml`) with unknown keys rejected. Environment placeholders of the form `${VAR}` must resolve at load time; otherwise configuration loading raises a `ConfigError`. This discipline induces the following basic hypothesis: all run-defining functions are measurable with respect to the configuration and the environment map.

Let $\mathcal{C}$ denote the resolved configuration mapping, and let $\mathcal{E}$ denote the environment variables used for interpolation. The pipeline can be viewed as a deterministic operator

$$\mathcal{P} : (\mathcal{C}, \mathcal{E}) \mapsto \{\text{artifacts, metrics, trained model}\}, \quad (1)$$

where nondeterminism is restricted to explicitly controlled random seeds and to external I/O latencies. In particular, the configuration constrains the valid set of executions through hard checks, for example the split consistency constraint

$$\text{training.validation_split} = \text{preprocessing.train_test_split.validation_ratio}, \quad (2)$$

and the output-type constraint that snapshot training and evaluation require

$$\text{model.output.type} = \text{"two_head_intensity"}. \quad (3)$$

The global control flow includes optional diagnostics and optional hyperparameter optimization (HPO), followed by final training and evaluation in “production” mode. Figure 1 depicts this orchestration.

2.2 Time discretization and sample geometry

The data source is a time-series database queried over a configured range $[t_{\min}, t_{\max})$ in UTC date-time. A cadence $\Delta > 0$ seconds defines the logical sampling grid. Let $t_i = t_{\min} + i\Delta$ be the i th grid time. The “visible window” length W_{vis} (seconds) defines the number of historical frames per sample,

$$T = \frac{W_{\text{vis}}}{\Delta}, \quad T \in \mathbb{N}, \quad (4)$$

which is enforced in configuration by requiring that W_{vis} be an integer multiple of Δ . The prediction horizon W_{hor} (seconds) similarly induces

$$H = \frac{W_{\text{hor}}}{\Delta}, \quad H \in \mathbb{N}, \quad (5)$$

where H is the number of forward steps used to construct the labels.

Samples are anchored at grid-aligned times for which both a complete past window of length T and a complete future window of length H exist. In the streaming implementation, a buffer maintains the last T frames; once H future frames are available after an anchor, the corresponding label is emitted.

2.3 Order book representation and hybrid depth aggregation

The system supports a hybrid depth representation that retains high resolution near the best quotes while compressing deeper levels into aggregated bins. Let L be the configured maximum depth levels per side. The hybrid configuration chooses integers L_{raw} and B such that

$$0 < L_{\text{raw}} < L, \quad 0 < B, \quad L_{\text{raw}} + B \leq L. \quad (6)$$

For each side (bid or ask), the top L_{raw} levels are kept as-is; the remaining $L - L_{\text{raw}}$ levels are partitioned into B bins defined by integer boundaries $\{e_b\}_{b=0}^B$ with $e_0 = L_{\text{raw}}$ and $e_B = L$. In the equal-width case, boundaries are approximately linear in level index; in the log-spaced case, they are approximately logarithmic to preserve more granularity closer to the market.

Within a bin b with indices $\ell \in [e_b, e_{b+1})$, the aggregated quantity is

$$\bar{Q}_b = \sum_{\ell=e_b}^{e_{b+1}-1} Q_\ell, \quad (7)$$

and the aggregated price is the volume-weighted average price (VWAP) whenever $\bar{Q}_b > 0$,

$$\bar{P}_b = \frac{\sum_{\ell=e_b}^{e_{b+1}-1} P_\ell Q_\ell}{\sum_{\ell=e_b}^{e_{b+1}-1} Q_\ell}, \quad (8)$$

with a fallback to an unweighted mean over $\{P_\ell\}$ if $\bar{Q}_b = 0$.

The resulting hybrid snapshot for one asset at one time is a matrix in $\mathbb{R}^{(L_{\text{raw}}+B) \times 4}$ whose columns contain bid price, bid quantity, ask price, and ask quantity, consistent with `preprocessing/depth_aggregator.py`.

2.4 Multi-asset alignment and confidence masking

The configuration may specify a target asset and a set of correlated assets. All non-target assets are aligned to the target asset’s timestamp grid using either linear interpolation or bucketing. Denote by $v(t)$ the representation value (either the hybrid matrix flattened or a top-of-book vector) for a correlated asset at time t , and let $\{t_k\}$ be its observed timestamps. For a target time τ that lies between two observed times $t_k \leq \tau \leq t_{k+1}$, the interpolation mode computes

$$\alpha(\tau) = \frac{\tau - t_k}{t_{k+1} - t_k}, \quad \tilde{v}(\tau) = (1 - \alpha(\tau)) v(t_k) + \alpha(\tau) v(t_{k+1}), \quad (9)$$

provided that the gap $t_{k+1} - t_k$ does not exceed a configured maximum gap; otherwise a missing-data policy applies. In bucketing mode, each observed time is assigned to a cadence bucket of width Δ and the nearest observation within a tolerance is selected.

Alignment also propagates a scalar confidence $c(t) \in [0, 1]$ per asset and time, which is linearly interpolated under interpolation and forward-filled under forward-fill policies. When enabled, the pipeline appends a confidence mask “channel” by broadcasting $c(t)$ across the spatial dimensions of the input tensor. Concretely, if the per-asset sequence tensor for A assets is $X \in \mathbb{R}^{T \times H \times W \times A}$, the augmented tensor is

$$X' = \text{concat}_{\text{channels}}(X, M), \quad M_{t,h,w,a} = c_a(t), \quad (10)$$

so that the model can down-weight or learn to ignore frames with low observational support.

2.5 Snapshot tensor construction and frame-store materialization

Training operates on fixed-shape tensors that are built from aligned per-asset records over a visible window of T steps. For a single asset $a \in \{1, \dots, A\}$ and time t , the hybrid order book representation is a matrix

$$S^{(a)}(t) \in \mathbb{R}^{L_{\text{eff}} \times 4}, \quad L_{\text{eff}} = L_{\text{raw}} + B, \quad (11)$$

whose four columns store (**bid_price**, **bid_qty**, **ask_price**, **ask_qty**) at each effective level. For the top-of-book representation, the system maps these four scalars into a small spatial patch whose height and width are configured; in the default layout, the entries are placed as

$$F^{(a)}(t) = \begin{bmatrix} \text{bid_price}(t) & \text{bid_qty}(t) \\ \text{ask_price}(t) & \text{ask_qty}(t) \end{bmatrix}. \quad (12)$$

For either representation, a multi-asset frame tensor is formed by stacking assets along the channel axis. In hybrid mode, the frame tensor has shape $\mathbb{R}^{L_{\text{eff}} \times 4 \times A}$ and satisfies

$$F(t)[:, :, a] = S^{(a)}(t), \quad (13)$$

while in top-of-book mode $F(t) \in \mathbb{R}^{H \times W \times A}$ is defined analogously. A sample anchored at time t_0 consists of the visible window of frames,

$$X_{\text{base}}(t_0) = (F(t_0 - (T - 1)\Delta), \dots, F(t_0)) \in \mathbb{R}^{T \times H \times W \times A}, \quad (14)$$

where $(H, W) = (L_{\text{eff}}, 4)$ in hybrid mode and (H, W) are the configured patch dimensions in top-of-book mode.

If confidence masking is enabled, an additional mask tensor $M(t_0) \in \mathbb{R}^{T \times H \times W \times A}$ is created by broadcasting the per-asset confidence scalars across spatial dimensions,

$$M(t_0)_{\tau, h, w, a} = c_a(t_0 - (T - 1 - \tau)\Delta), \quad (15)$$

and the masked input becomes $\text{concat}_{\text{channels}}(X_{\text{base}}(t_0), M(t_0))$, doubling the asset-channel count.

Auxiliary features are computed at the anchor and include (i) derived order book features and momentum features and (ii) temporal encodings. The resulting vector $u(t_0) \in \mathbb{R}^{d_{\text{aux}}}$ is either consumed by a separate model branch or, when `integration_mode=concat_channels`, broadcast into a tensor

$$U(t_0)_{\tau, h, w, j} = u_j(t_0), \quad (16)$$

and appended as additional channels. The final sample tensor therefore has channel dimension

$$C = A + \mathbf{1}\{\text{include_mask_channel}\}A + d_{\text{aux}}, \quad (17)$$

which matches the reference materialization procedure in `training/snapshot_dataset.py:_materialize_frame`.

To avoid storing duplicated visible windows for each sample, the snapshot builder uses a frame-store representation (**frame_store_v1**) in which all frames in a chunk are stored once as an array **frames_base** $\in \mathbb{R}^{N_{\text{frames}} \times H \times W \times A}$ together with per-frame confidence values **frames_confidence** $\in [0, 1]^{N_{\text{frames}} \times A}$. Each sample stores an anchor index $s_i \in \{0, \dots, N_{\text{frames}} - 1\}$ and an auxiliary vector $u_i \in \mathbb{R}^{d_{\text{aux}}}$. The sample tensor is reconstructed by slicing contiguous windows,

$$X_{\text{base}}^{(i)}[\tau] = \text{frames_base}[s_i - (T - 1) + \tau], \quad \tau = 0, \dots, T - 1, \quad (18)$$

and broadcasting the corresponding confidence and auxiliary values in the same manner described above.

2.6 Label construction: two-head intensity

Let $m(t)$ denote the mid-price at time t (computed as half the sum of best bid and best ask when both are positive). For an anchor time t_0 , the system forms the future window $\{m(t_0 + \Delta), \dots, m(t_0 + H\Delta)\}$ and computes relative moves in percent,

$$r(h) = 100 \frac{m(t_0 + h\Delta) - m(t_0)}{m(t_0)} \quad (h = 1, \dots, H). \quad (19)$$

The upward and downward summary statistics are

$$r^\uparrow(t_0) = \max_{1 \leq h \leq H} r(h), \quad r^\downarrow(t_0) = \min_{1 \leq h \leq H} r(h). \quad (20)$$

Given ordered boundaries $0 < b_1 < \dots < b_K$ (percent), the upward intensity class is

$$y^\uparrow(t_0) = \min\{k \in \{0, \dots, K\} : \max(r^\uparrow(t_0), 0) \leq b_{k+1}\}, \quad (21)$$

with the convention that $b_{K+1} = +\infty$; the downward class uses the nonnegative magnitude $\max(-r^\downarrow(t_0), 0)$ and the same binning rule. This corresponds exactly to `training/snapshot_dataset.py:_compute`

2.7 Normalization and sample weighting

Feature normalization follows a fit/transform discipline, with statistics computed in a streaming pass over the snapshot dataset (`training/snapshot_dataset.py:compute_normalization_stats`). Let $x \in \mathbb{R}^d$ denote the flattened input features for a sample, restricted to the non-mask subset of channels (mask channels, when enabled, are excluded from the statistics and left unchanged). In snapshot mode, only `min_max` and `standard` scaling are implemented; selecting `robust` raises a `ConfigError` even though robust scaling exists for non-snapshot paths in `preprocessing/normalizer.py`.

Let m_j and M_j denote feature-wise minima and maxima, and let μ_j and σ_j denote feature-wise means and standard deviations. The snapshot-mode transforms are

$$\text{min-max: } \hat{x}_j = \frac{x_j - m_j}{M_j - m_j}, \quad (22)$$

$$\text{standard: } \hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}. \quad (23)$$

Denominators are guarded against zero by replacing zeros with one. When `fit_on_train_only=true`, statistics are computed on the training split and reused for validation/test; otherwise, separate statistics are computed per split and cached. In the current snapshot implementation, statistics are global over all non-mask features (the configuration flag `preprocessing.normalization.per_asset` is not applied in snapshot mode).

Training uses per-sample weights for each head. For a sample anchored at time t_0 , a duty-cycle weight is computed as the fraction of observed frames in its visible window,

$$w_{\text{duty}}(t_0) = \frac{1}{T} \sum_{\tau=t_0-(T-1)\Delta}^{t_0} \mathbf{1}\{\text{frame at } \tau \text{ observed}\}. \quad (24)$$

Optionally, a time-decay weight emphasizes recency using a half-life λ (days),

$$w_{\text{decay}}(t_0) = \exp\left(-\log(2) \frac{\text{age_days}(t_0)}{\lambda}\right), \quad (25)$$

and optional class weights map labels to scalars $w_{\text{class}}^{\uparrow}(y^{\uparrow})$ and $w_{\text{class}}^{\downarrow}(y^{\downarrow})$. The final per-head sample weights are the products

$$w^{\uparrow}(t_0) = w_{\text{duty}}(t_0) w_{\text{decay}}(t_0) w_{\text{class}}^{\uparrow}(y^{\uparrow}(t_0)), \quad w^{\downarrow}(t_0) = w_{\text{duty}}(t_0) w_{\text{decay}}(t_0) w_{\text{class}}^{\downarrow}(y^{\downarrow}(t_0)), \quad (26)$$

matching the generator logic in `training/snapshot_dataset.py`.

2.8 Model architecture and objective

The baseline model is a Keras CNN+LSTM network (`models/cnn_lstm_multiclass.py`) that applies a shared per-frame convolutional feature extractor, then an LSTM over time, then dense layers, and finally two softmax heads. Long-term context features can optionally add a second input branch whose output is concatenated with the short-term embedding.

Let $p^{\uparrow}(\cdot | x)$ and $p^{\downarrow}(\cdot | x)$ denote the predicted distributions, and let $y^{\uparrow}, y^{\downarrow}$ be the class labels. The per-sample loss is the sum of cross-entropies,

$$\ell(x) = -\log p^{\uparrow}(y^{\uparrow} | x) - \log p^{\downarrow}(y^{\downarrow} | x), \quad (27)$$

and training minimizes the weighted empirical risk, using the per-head weights $(w^{\uparrow}, w^{\downarrow})$ when enabled.

2.9 Evaluation, calibration, and backtesting

Evaluation computes classification metrics per head and may perform calibration analysis. For a head with one-hot labels $Y \in \{0, 1\}^{n \times K}$ and predicted probabilities $P \in [0, 1]^{n \times K}$, the implemented Brier score is

$$\text{Brier}(P, Y) = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K (P_{ik} - Y_{ik})^2, \quad (28)$$

and the expected calibration error (ECE) is computed over bins on the flattened probabilities (not only on top-1 confidence). Temperature scaling optionally fits a scalar $T > 0$ to logits $z_i \in \mathbb{R}^K$ by minimizing the negative log-likelihood

$$T^* = \arg \min_{T \in [T_{\min}, T_{\max}]} -\frac{1}{n} \sum_{i=1}^n \log \text{softmax}(z_i/T)_{y_i}, \quad (29)$$

which is solved by bounded scalar minimization.

The evaluation pipeline can also compute temporal degradation metrics by slicing time into windows, and it can run a simplified backtest that converts two-head outputs into a trading signal according to a configured strategy (e.g., net intensity or thresholding).

3 Functionalities

This section maps the system’s major modules to concrete behaviors and, where appropriate, to their mathematical definitions.

3.1 Configuration loading, schema validation, and semantic constraints

The configuration loader (`utils/config_loader.py`) enforces three invariants:

- **Schema adherence:** required sections and required keys must be present; unknown keys are rejected (`strict_unknown_keys=true` in `config/validation_schema.yaml`).
- **Environment interpolation:** placeholders of the form `${VAR}` must resolve at load time; otherwise load fails with `ConfigError`.
- **Semantic checks:** configuration must satisfy run-time constraints, including the two-head intensity contract (`targets.labeling.scheme` and `model.output.type`) and consistency checks such as `model.output.num_classes = len(targets.price_classes.boundaries)+1`.

In particular, automatic boundary selection (`targets.price_classes.boundaries="auto"`) and automatic class count derivation (`model.output.num_classes="auto"`) are only permitted under `two_head_intensity`.

3.2 Snapshot context, configuration hash, and directory resolution

Snapshots are derived datasets stored on disk under `snapshot.directory`. Their purpose is to make training restartable and reproducible without re-querying the database.

Let $\mathcal{C}$ be the resolved configuration. The snapshot store (`training/snapshot_store.py`) defines a configuration subset operator $S(\mathcal{C})$ that retains only snapshot-relevant keys (data asset pairs, time range, order-book representation, temporal features, alignment; target horizon/window and labeling; preprocessing normalization + feature engineering; and model input/output descriptors). A deterministic hash is computed as

$$h(\mathcal{C}) = \text{SHA256}(\text{JSON}(S(\mathcal{C}))), \quad (30)$$

where JSON serialization uses sorted keys and compact separators.

Snapshot name resolution follows `snapshot.name`:

- If `name="auto"`, the resolved name is `<root_name>_<hash8>_<target_asset>_<start_date>_<end_date>` with non-alphanumeric characters sanitized to `-`.
- Otherwise, `name` is used verbatim.

On configuration mismatch, `snapshot.on_config_mismatch` admits only two policies (enforced by `ConfigError`):

- `"error"`: refuse to reuse an existing snapshot directory whose manifest hash differs from $h(\mathcal{C})$.
- `"create_new"`: create a new sibling snapshot by suffixing the resolved name with `_<hash8>`.

3.3 Snapshot eviction policy

To bound disk usage, `training/snapshot_store.py:maybe_evict_snapshots` removes old snapshot directories beyond `snapshot.max_snapshots` by sorting candidates by `created_at` (falling back to manifest modification time) and deleting the oldest, never deleting the currently resolved snapshot.

3.4 Snapshot manifest schema and invariants

Each snapshot directory contains a `manifest.json` written atomically (`write-then-os.replace`). The manifest is a JSON object with stable top-level keys:

- `version`: manifest schema version (currently 1).
- `snapshot_name`, `root_name`, `created_at` (UTC ISO8601).
- `config_hash` (equals $h(C)$) and `config_snapshot` (equals $S(C)$).
- `time_range` and `assets`.
- `chunks`: list of chunk entries (see below).
- `series`: metadata and chunked target-asset series files (timestamps, mid-prices, volumes) used by long-term features and auto boundary fitting.
- `normalization_stats`: cached normalization-stat descriptors written by the training pipeline.
- `complete`: boolean indicating whether the chunk build finished.

3.5 Chunk boundaries and ingestion semantics

Order-book rows are streamed in physical time chunks of `data.ingestion.chunk_hours` (`data/greptime_client`). For a given date interval, `_generate_time_chunks` emits non-overlapping chunks whose endpoints are expressed as YYYY-MM-DD HH:MM:SS strings; the final chunk uses an inclusive end-of-day boundary. If multi-database ingestion is enabled (`data.multi_database.enabled=true`), non-overlapping per-connection date ranges are intersected with the global range and processed sequentially (“time split”).

3.6 Snapshot chunk storage formats

The snapshot loader supports three chunk storage formats for backward compatibility, identified by `entry["format"]` in the manifest:

- `npz`: a single `.npz` containing `x`, `y_up`, `y_down`, `anchor_ts`, and `duty_cycle`.
- `numpy_shards_v1`: separate `.numpy` arrays for `x`, `y_up`, `y_down`, `anchor_ts`, `duty_cycle`.
- `frame_store_v1`: a “frames once, anchors many” representation that stores per-frame arrays and per-sample metadata.

The current snapshot builder (`training/snapshot_dataset.py:_build_snapshot_chunks`) writes `frame_store_v1`.

3.6.1 Frame store format: `frame_store_v1`

For a chunk with A assets, visible window length T (`window_steps`), spatial dimensions (H, W) , and auxiliary dimension d_{aux} , the persisted arrays are:

$$\text{frames_base} \in \mathbb{R}^{N_{\text{frames}} \times H \times W \times A}, \quad (31)$$

$$\text{frames_confidence} \in [0, 1]^{N_{\text{frames}} \times A}, \quad \text{frames_observed} \in \{0, 1\}^{N_{\text{frames}} \times A}, \quad (32)$$

$$\text{frames_ts} \in \mathbb{Z}^{N_{\text{frames}}}, \quad (33)$$

$$\text{anchor_local_idx} \in \mathbb{Z}^{N_{\text{samples}}}, \quad \text{aux} \in \mathbb{R}^{N_{\text{samples}} \times d_{\text{aux}}}, \quad (34)$$

$$\text{y_up}, \text{y_down} \in \{0, \dots, K-1\}^{N_{\text{samples}}}, \quad \text{anchor_ts} \in \mathbb{Z}^{N_{\text{samples}}}, \quad (35)$$

$$\text{duty_cycle} \in [0, 1]^{N_{\text{samples}}}. \quad (36)$$

The reconstructed model input has shape $(N_{\text{samples}}, T, H, W, C)$ with

$$C = A + 1\{\text{include_mask_channel}\} A + d_{\text{aux}}. \quad (37)$$

The per-sample base window is sliced from `frames_base` using `anchor_local_idx`:

$$X_{\text{base}}^{(i)}[\tau] = \text{frames_base}[s_i - (T-1) + \tau], \quad s_i = \text{anchor_local_idx}[i], \quad \tau = 0, \dots, T-1. \quad (38)$$

If mask channels are enabled, the corresponding mask tensor is built by broadcasting per-frame confidence values across spatial dimensions and concatenating along the channel axis. Auxiliary features are broadcast analogously (`training/snapshot_dataset.py:_materialize_frame_store_v1_x`).

3.6.2 Chunk overlap frames and anchor mapping

To permit samples whose visible window crosses a chunk boundary, each `frame_store_v1` chunk stores a prefix of up to $T-1$ tail frames from the previous chunk. Let $\tilde{N}$ be the number of “core” frames derived from the current chunk’s streamed rows; the stored frame count is

$$N_{\text{frames}} = \tilde{N} + N_{\text{overlap}}, \quad N_{\text{overlap}} = \min(T-1, \text{available previous-tail frames}). \quad (39)$$

Anchor timestamps are mapped to local indices only for core frames; the mapping is stored implicitly via `anchor_local_idx` constructed from `anchor_local_idx_by_ts`. This implies the invariant that for every sample i , the slice start index $s_i - (T-1)$ must be non-negative and the end index s_i must be strictly less than N_{frames} ; violations raise `ConfigError` during materialization.

3.7 Gap handling, alignment, and confidence semantics

Streaming snapshot construction uses a two-stage missing-data policy.

- **Per-asset gap handling (`GapHandler`):** within each asset stream, observed snapshots are optionally augmented with synthetic snapshots when gaps are between one cadence step and `alignment.max_gap_seconds`. Under `handle_gaps="forward_fill"`, features are copied from the previous observed snapshot; under `handle_gaps="interpolate"`, features are linearly interpolated between surrounding observations. Synthetic snapshots are marked `observed=false`.
- **Multi-asset alignment:** correlated assets are aligned to the target asset’s grid using either linear interpolation or bucket selection (`data.asset_pairs.alignment.method`). When `include_mask_channel=true`, a per-asset confidence scalar is propagated and later broadcast into mask channels.

Confidence is a scalar $c \in [0, 1]$ derived from gap size g and the configured maximum gap G as $c = \max\{0, 1 - g/G\}$ (with additional decay inside a multi-step gap), ensuring that synthetic frames are down-weighted by the model.

The per-sample duty cycle stored in `duty_cycle` is defined on the target asset only:

$$w_{\text{duty}}(t_0) = \frac{1}{T} \sum_{\tau=0}^{T-1} \mathbf{1}\{\text{target frame at } t_0 - (T - 1 - \tau)\Delta \text{ is observed}\}. \quad (40)$$

3.8 Split semantics, cached normalization statistics, and transform application

Snapshot training uses chronological splitting by global sample index (`training/pipeline.py`). Normalization statistics are cached as `normalization_stats_<split>.npz` under the snapshot directory and recorded in the manifest under `normalization_stats[split]`. Statistics are computed by streaming over the corresponding index range and flattening all non-mask channels; mask channels are excluded and left unchanged.

In snapshot mode, normalization statistics support only `min_max` and `standard` methods (`training/snapshot_`). The configuration flag `preprocessing.normalization.per_asset` is not applied in snapshot mode; statistics are global over all non-mask features.

3.9 Coarse class balancing (undersampling) and class weights

Two orthogonal mechanisms address imbalance:

- **Undersampling (pre-training filter):** when enabled, `training/sample_balancing.py` selects a subset of training indices to match a target class distribution. A single balancing label y_{bal} is derived from the two-head labels via

$$y_{\text{bal}} = \max(y^{\uparrow}, y^{\downarrow}) \text{ (max intensity); } y_{\text{bal}} = y^{\uparrow} \text{ (up); } y_{\text{bal}} = y^{\downarrow} \text{ (down),} \quad (41)$$

depending on `labeling_criteria`. Per-class keep counts are computed by scaling the target distribution to the largest feasible total under availability constraints, then selecting samples by `selection_policy` (uniform time, random, or k-means). Hard safeguards enforce `min_samples_after_balance` and `min_fraction_after_balance`.

- **Class weights (loss reweighting):** `training/class_weights.py` computes inverse-frequency weights

$$w_{\text{class}}[c] = \frac{n}{K \text{ count}[c]}, \quad (42)$$

with zero-count classes assigned the maximum weight among non-zero classes.

The training generator always applies duty-cycle weights and optionally multiplies exponential time-decay and per-head class weights (`training/snapshot_dataset.py:build_training_generator`).

3.10 Long-term context features (optional) and dual-input contract

When `model.long_term.enabled=true`, the pipeline computes long-term features for every sample from the snapshot series files (`training/long_term_context.py`). The computation is cached under the snapshot directory as `anchor_timestamps.npy` (concatenated per-chunk `anchor_ts`) and `long_term_features.npz` with a cache hash that includes the snapshot config hash, cadence, and long-term feature configuration.

Long-term feature definitions are implemented in `preprocessing/long_term_features.py` and explicitly exclude the anchor day from the lookback window to avoid lookahead. The training pipeline enforces a dual-input contract: long-term enabled implies the model must accept exactly two inputs, and long-term disabled implies the model must not expect a second input.

3.11 Sequential training over rolling windows (optional)

Sequential training partitions the configured date interval into consecutive windows of `training.sequential_train` (inclusive, day-granularity). Resume state is stored under `<snapshot.directory>/_sequential_resume/` as a JSON state file and a Keras model file keyed by a hash of the window list plus run identifiers (`training/pipeline.py`). Completed window snapshots can be cleaned up when configured.

3.12 Runtime device selection, mixed precision, and distributed training

Runtime configuration (`main.py`) selects `cpu` vs `gpu` execution, sets `CUDA_VISIBLE_DEVICES` when pinning GPUs, optionally configures `TF_GPU_ALLOCATOR=cuda_malloc_async`, and can enable mixed precision.

Distributed training is opt-in via `training.runtime.distributed` and currently supports `tf.distribute.MirroredStrategy` (`training/distributed.py`). If enabled, the global batch size equals the configured per-replica batch size times the number of replicas, and the Python generator is wrapped as a `tf.data.Dataset` with an explicit output signature (with or without long-term features).

3.13 Hyperparameter optimization (Optuna) and regime safeguards

Optuna-based HPO (`models/hyperparameter_tuning.py`) samples:

- CNN per-layer filter counts (`search_space.cnn[*].filters`).
- LSTM per-layer unit counts (`search_space.lstm[*].units`).
- Learning rate (`search_space.learning_rate`, optionally log-scaled).
- Batch size (`search_space.batch_size` as integer range or categorical set).

Sampled parameters are applied directly into the configuration tree before invoking the training pipeline.

Parallel HPO runs independent worker processes pinned to resources `"cpu"` or `"gpu:<id>"`. Each worker disables distributed training and sets GPU visibility to isolate resource usage. “Regime” safeguards optionally retry on OOM with batch-size backoff to a configured floor, clamp requested batch sizes to a persisted “safe envelope” learned from past successful trials, and run an RSS watchdog that can terminate or fall back to fewer workers if memory usage exceeds a configured limit.

3.14 Evaluation, calibration, and backtesting definitions

Evaluation (`evaluation/evaluator.py`) computes per-head accuracy and macro-averaged precision/recall/F1 from the confusion matrix.

Calibration analysis uses `evaluation/calibration.py`. ECE is computed on flattened probabilities (not top-1 confidence): for bins $B_b \subset [0, 1]$ and flattened vectors $p \in [0, 1]^{nK}$, $y \in \{0, 1\}^{nK}$,

$$\text{ECE} = \sum_b \frac{|B_b|}{nK} |\mathbb{E}[p \mid p \in B_b] - \mathbb{E}[y \mid p \in B_b]|. \quad (43)$$

Post-hoc temperature scaling fits separate temperatures for up and down heads by minimizing the negative log-likelihood over a configured fit range; when the model emits probabilities rather than logits, the implementation uses a log-probability “logit proxy” $z = \log p$ so that `softmax(z/T)` remains a valid calibrated distribution.

Backtesting (`evaluation/backtesting.py`) converts head probabilities into signals. For an intensity threshold k_0 ,

$$p_{\uparrow} = \sum_{k \geq k_0} p_k^{\uparrow}, \quad p_{\downarrow} = \sum_{k \geq k_0} p_k^{\downarrow}. \quad (44)$$

The `net_intensity` strategy goes long if $p_{\uparrow} > p_{\downarrow}$ and short if $p_{\downarrow} > p_{\uparrow}$, with confidence $|p_{\uparrow} - p_{\downarrow}|$. The `threshold` strategy trades only if *either* probability exceeds a configured threshold, breaking ties by choosing the larger. Trades are entered at time t and exited after H steps; PnL is computed in percent with a round-trip transaction cost of `2 * transaction_cost_pct`.

3.15 Observability server and run-state semantics

The observability server (`observability/server.py`) is a stdlib `http.server`-based process that serves an HTMX dashboard and JSON APIs, guarded by Basic Auth (credentials provided via environment variables). Run state is recorded by the pipeline into a SQLite-backed run-state file (`observability/run_state.py`) with monotone stage progression (`idle` $\rightarrow$ `initializing` $\rightarrow$ `diagnostics` $\rightarrow$ `snapshot_build` $\rightarrow$ `trial` $\rightarrow$ `training` $\rightarrow$ `evaluation`). The server exposes:

- health checks (`/healthz`),
- UI pages (`/ui*`) and configuration browsing/editing (`/ui/config` and JSON config APIs),
- run state and history APIs (`/api/run_state`, `/api/history`, `/api/hpo`),
- logs (`/api/logs`, `/ui/logs`),
- Prometheus-style metrics (`/metrics`),
- run control endpoints that start/stop `main.py` under the server with `RUN_STATE_PATH` and `RUN_LOG_PATH` exported.

4 Build and Rendering Instructions

This document is self-contained and can be rendered from `Notes/tex/`.

- **LaTeX build (pdflatex):** run `pdflatex cnn_lstm_feature_representation.tex` twice to resolve references.
- **LaTeX build (tectonic):** run `tectonic -X compile cnn_lstm_feature_representation.tex` (it automatically reruns as needed).

- **Mermaid figures (optional):** if a rendered image is present (`figures/*.pdf` or `figures/*.png`), it is included; otherwise the build falls back to embedding the Mermaid source (`figures/*.mmd`) via the `\MermaidFigure` macro.
- **Optional render step (curl + Kroki):** run `bash render_mermaid_figures.sh` from `Notes/tex/` to render `figures/*.mmd` into `figures/*.png`.
- **Optional render step (Node + Mermaid CLI):** with Node.js installed, install `@mermaid-js/mermaid-cli` and render each `.mmd` to `.png` or `.pdf` under `Notes/tex/figures/` (e.g. `mmdc -i pipeline_flow.mmd -o pipeline_flow.png`).

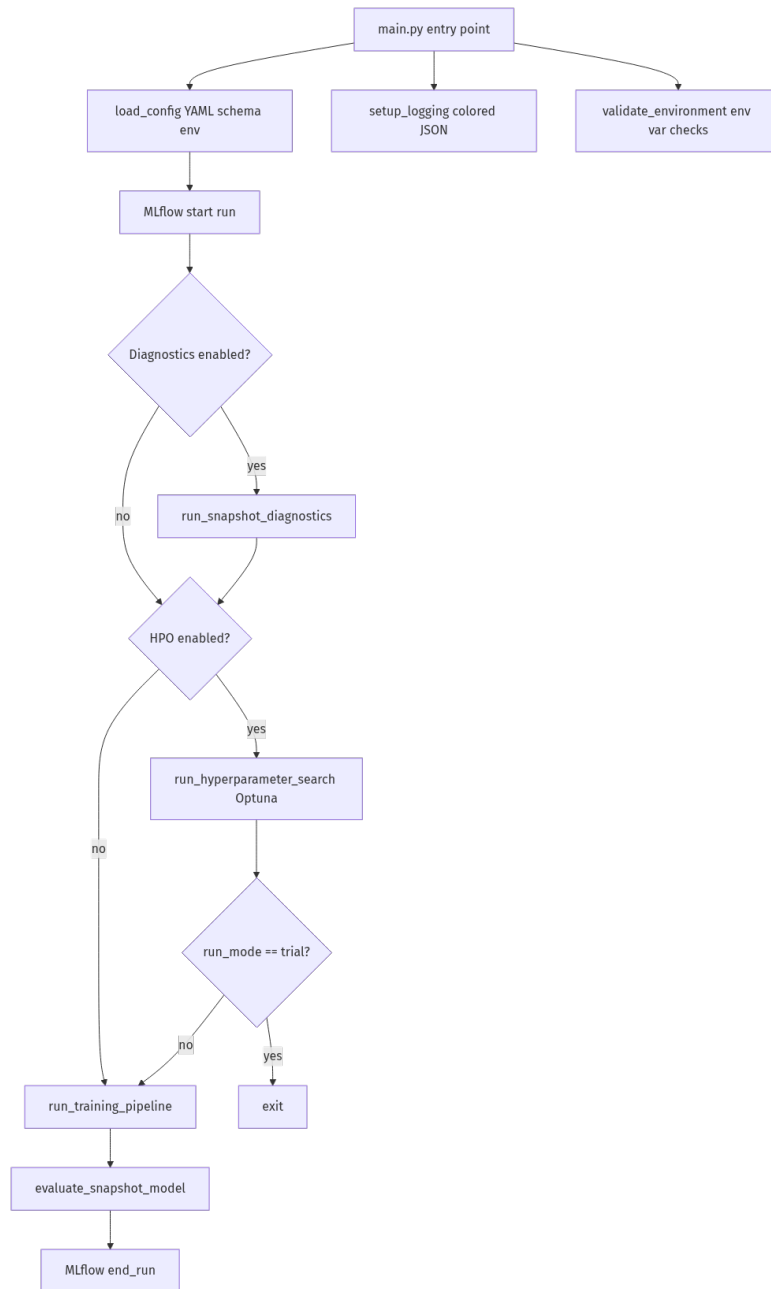


Figure 1: Top-level pipeline flow (configuration load, optional diagnostics and HPO, training, evaluation, MLflow lifecycle). (Mermaid source: `figures/pipeline_flow.mmd`).

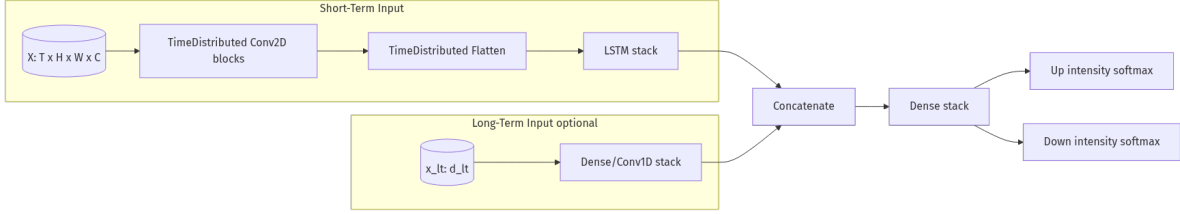


Figure 2: Model architecture: short-term CNN+LSTM with optional long-term branch and two softmax heads. (Mermaid source: `figures/model_architecture.mmd`).

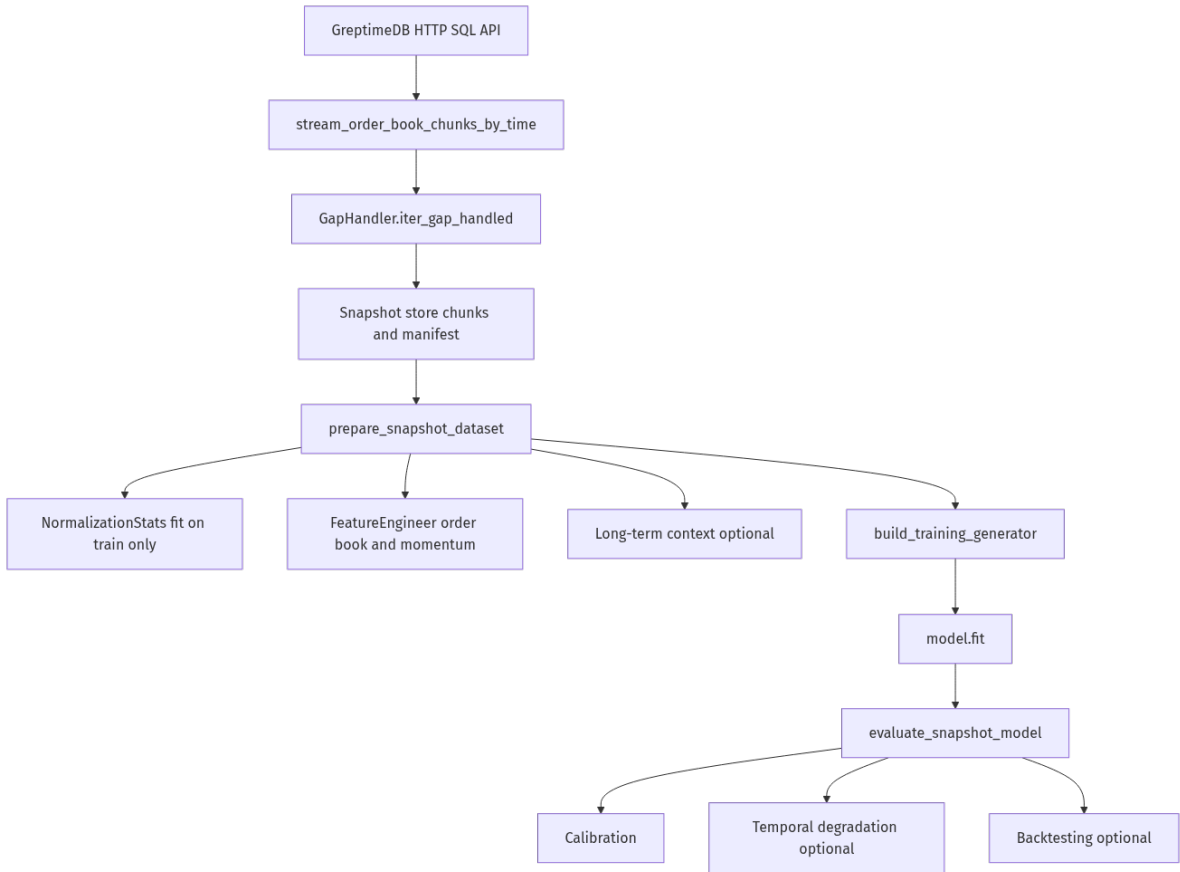


Figure 3: Data flow from GreptimeDB ingestion through snapshot materialization, training, evaluation, and optional calibration/backtesting. (Mermaid source: `figures/data_flow.mmd`).